

Project Structure

```
Lyricea/
+-- data/
|   +-- history.json      # Search history (auto-created)
+-- docs/                 # Documentation sources
+-- scripts/
|   +-- dev.js            # Concurrent dev server launcher
+-- server/
|   +-- index.js          # Express app & routes
|   +-- lyrics.js         # Search, fetch, translate logic
|   +-- db.js             # JSON file persistence
+-- src/
|   +-- App.jsx           # Root state & orchestration
|   +-- main.jsx          # React entry point
|   +-- components/
|   |   +-- SearchBar.jsx # Autocomplete search
|   |   +-- LyricsDisplay.jsx # Bilingual lyrics + export
|   |   +-- HistoryPanel.jsx # Sidebar history list
|   +-- utils/
|       +-- pdfExport.js  # Client-side A1 PDF generation
+-- index.html
+-- vite.config.js
+-- package.json
```

Development Setup

Prerequisites

- * Node.js 18+
- * npm

Commands

Command	Description
npm install	Install dependencies
npm run dev	Start Vite + Express with hot reload
npm run dev:client	Vite only
npm run dev:server	Express with --watch only
npm run build	Build frontend to dist/
npm start	Run production server
npm run preview	Preview built frontend via Vite

Vite Proxy

vite.config.js proxies /api to http://localhost:3001. Also sets resolve.preserveSymlinks: true for network drive compatibility (UNC paths on Windows mapped drives).

Frontend

State Management

All application state lives in App.jsx-no Redux, Context, or router.

State	Type	Purpose	
currentSong	object \	null	Active song with pairs,
loading	boolean	Blocks UI during fetch	
error	string \	null	Error banner message
history	array	Sidebar items from GET	
historyOpen	boolean	Toggle history panel visibility	

Component Responsibilities

SearchBar.jsx

- * Local state: query, suggestions, open, activeIndex, searching
- * Debounced search: 280ms delay, minimum 2 characters
- * Calls GET /api/search?q=...
- * Keyboard: ArrowUp/Down, Enter, Escape
- * On select: calls onSelect(song) prop with { title, artist, album, artworkUrl }

LyricsDisplay.jsx

- * Renders song header (artwork, title, artist, album)
- * Maps pairs to Spanish/English blocks
- * exportLyricsPdf() on button click (client-side only)

HistoryPanel.jsx

- * Presentational list; callbacks: onSelect(id), onDelete(id), onClear()
- * Highlights activeId matching current song's historyId

Data Flow

```
SearchBar.onSelect(song)
  -> App.fetchLyrics(song)
  -> POST /api/lyrics
  -> setCurrentSong(data)
  -> loadHistory()

HistoryPanel.onSelect(id)
  -> App.loadFromHistory(id)
  -> GET /api/history/:id
  -> setCurrentSong (from cache, no re-translate)
```

Backend API

Base URL: /api ? Content-Type: application/json

GET /api/search?q={query}

Search songs via iTunes Search API.

Query params: q - search string (returns [] if < 2 chars)

Response:

```
{
  "results": [
    {
      "title": "Despacito",
      "artist": "Luis Fonsi",
      "album": "Vida",
      "artworkUrl": "https://...",
      "releaseDate": "2017-01-13T08:00:00Z"
    }
  ]
}
```

Deduplication key: title|artist (case-insensitive). Artwork upscaled from 100?100 to 300?300.

POST /api/lyrics

Fetch lyrics, translate, optionally save to history.

Request body:

```
{
  "title": "Despacito",
  "artist": "Luis Fonsi",
  "album": "Vida",
  "artworkUrl": "https://...",
  "save": true
}
```

Field	Required	Default	Notes
title	yes	-	Track name
artist	yes	-	Artist name
album	no	-	Stored in history
artworkUrl	no	-	Stored in history
save	no	true	Set false to skip history write

Success (200):

```
{
  "title": "Despacito",
  "artist": "Luis Fonsi",
  "album": "Vida",
  "artworkUrl": "https://...",
  "source": "lrclib",
  "spanishLyrics": "full raw text...",
  "pairs": [{ "spanish": "...", "english": "..."}],
  "historyId": 3
}
```

Errors: 400 missing fields ? 404 lyrics not found ? 500 server error

GET /api/history?limit={n}

Returns recent searches, newest first. Default limit: 50.

Response:

```
{
  "items": [
    {
      "id": 3,
      "title": "...",
      "artist": "...",
      "album": "...",
      "artwork_url": "...",
      "searched_at": "2026-07-01T12:00:00.000Z",
      "pairs": [{ "spanish": "...", "english": "..."}]
    }
  ]
}
```

Note: spanish_lyrics and raw english_lyrics are stripped from list responses; pairs is parsed from stored JSON.

GET /api/history/:id

Returns full cached entry including pairs. Does not re-fetch external APIs.

DELETE /api/history/:id

Removes one history entry by numeric ID.

DELETE /api/history

Clears all history and resets nextId to 1.

Static & SPA Fallback

After API routes, Express serves dist/ statically. Non-API GET requests fall back to dist/index.html.

Lyrics & Translation Pipeline

Implemented in server/lyrics.js.

Song Search (searchSongs)

```
iTunes Search API
-> filter trackName + artistName
-> dedupe by title|artist
-> return up to 12 results (lang=es_es)
```

Lyrics Fetch (fetchLyrics)

Sequential fallback chain:

1. LRCLIB direct - GET /api/get?track_name=&artist_name=
2. LRCLIB search - GET /api/search?q= -> exact title/artist match or first result
3. lyrics.ovh - GET /v1/{artist}/{title}

Synced lyrics timestamps stripped: `^\[d+:\d+\.\d+\]/g`

Translation (translateLyrics)

1. Split lyrics into non-empty lines
2. Batch lines in groups of 8, joined with `\n`
3. Call Google Translate unofficial endpoint per batch:

`https://translate.googleapis.com/translate_a/single?client=gtx&sl=es&tl=en&dt=t&q=...`

1. Split translated result by `\n` and zip with original lines into { spanish, english } pairs

Caveat: Batch translation may misalign lines if Google merges or splits differently than the input.

Combined Entry Point

`getLyricsWithTranslation(title, artist) -> { found, source, spanishLyrics, pairs } or { found: false, message }`

Data Storage

`server/db.js` - synchronous JSON file I/O.

File: `data/history.json`

Schema:

```
{
  "items": [
    {
      "id": 1,
      "title": "string",
      "artist": "string",
      "album": "string | null",
      "artwork_url": "string | null",
      "spanish_lyrics": "string",
      "english_lyrics": "string (JSON-serialized pairs array)",
      "searched_at": "ISO 8601 datetime"
    }
  ],
  "nextId": 2
}
```

Function	Behavior
<code>getHistory(limit)</code>	Sort by <code>searched_at</code> desc, slice
<code>getHistoryItem(id)</code>	Find by ID
<code>saveHistory(entry)</code>	Append with auto-increment ID
<code>deleteHistoryItem(id)</code>	Filter out ID

clearHistory()

Reset to empty store

Concurrency: No file locking; not safe for multi-process writes.

PDF Export

src/utils/pdfExport.js - runs entirely in the browser via jsPDF.

Setting	Value
Page size	A1 portrait (594 × 841 mm)
Spanish color	RGB(30, 100, 140)
English color	RGB(45, 110, 55)
Font	Helvetica (built-in)
Pagination	Auto addPage() when content exceeds margin

Filename: {sanitized_title}_{sanitized_artist}_lyrics.pdf

Configuration

Variable	Default	Description
PORT	3001	Express listen port
NODE_ENV	-	Not explicitly used; standard Node

No .env file is required. CORS is enabled for all origins (cors() default).

Production Build & Deployment

```
npm run build    # outputs to dist/
npm start       # Express serves dist/ + API on PORT
```

Hosting Checklist

1. Node.js 18+ runtime
2. Build command: npm install && npm run build
3. Start command: npm start
4. Persistent volume mounted at data/ if history must survive redeloys
5. Outbound HTTPS required for iTunes, LRCLIB, lyrics.ovh, Google Translate

Platform Notes

Platform	Fit	Persistence
Render / Railway	Excellent	Attach disk/volume for data/

Fly.io	Good	fly volumes for data/
VPS + PM2 + nginx	Full control	Local disk by default
Vercel / Netlify	Poor fit	Serverless ? long-running Express + file I/O

Example nginx reverse proxy

```
location / {
    proxy_pass http://127.0.0.1:3001;
    proxy_http_version 1.1;
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
}
```

External Dependencies & Risks

Service	Endpoint	Purpose	Risk
iTunes Search	itunes.apple.com/search	Autocomplete	Rate limits
LRCLIB	lrclib.net/api	Primary lyrics	Availability varies
lyrics.ovh	api.lyrics.ovh	Fallback lyrics	Slow / intermittent
Google Translate	translate.googleapis.com	Translation	Rate limits, ToS, IP blocking

Recommendations for production:

- * Replace unofficial Google Translate with DeepL or Google Cloud Translation API
- * Add fetch timeouts and retry with backoff
- * Rate-limit /api/lyrics per IP
- * Consider SQLite/Postgres instead of JSON file for concurrent access

Extension Points

Goal	Suggested approach
Better translations	Swap translateChunk() for official API in
Persistent hosting storage	SQLite via better-sqlite3; minimal
User accounts	Add auth middleware; scope history by
More languages	Parameterize sl/tl in translate calls + UI
Caching lyrics	Redis or on-disk cache keyed by `title` `artist`
Docker	FROM node:20-alpine, npm ci, npm run

Dependencies

Runtime

- * express ^4.21 - HTTP server

- * cors ^2.8.5 - Cross-origin middleware
- * react / react-dom ^18.3 - UI
- * jspdf ^2.5 - PDF generation (client)

Dev

- * vite ^6.0 - Bundler & dev server
- * @vitejs/plugin-react - JSX support
- * concurrently - Listed but dev uses custom scripts/dev.js

Lyrica Technical Documentation - v1.0